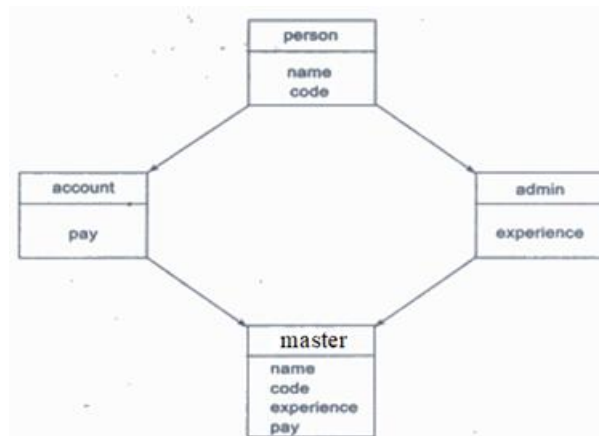


OOP Programming Assignment (Date: 07-April 2026)

1. Create a base class called shape. Use this class to store two double type values that could be used to compute the area of the figures. Derive two specific classes called triangle and rectangle from the base shape. Add to the base class, a member function `get_data()` to initialize the base class data members and another member function `display_area()` to compute and display the area of the figures.
 - a. Using these three classes, design a program that will accept dimensions of a triangle or a rectangle interactively, and display the area.
2. Extend the above program to display the area of the circle using a circle class. Remember for circle you need only one value radius.
3. Consider the class network below, the class master derives information from both admin and account classes which in turn derive information from the class person. Define all the four classes and write a program to create, update and display the information contained in master objects.



4. Design a class Polar which describes a point in the polar coordinates using radius and angle.
 - a. Define a member function to convert the point in rectangular coordinates.
 - b. Overload the operator `+` to add two polar objects.
 - c. Using friend function to overload insertion and extraction operators.
5. Define a class String, that holds the string variable as a character array. Use overloaded `==` operator to compare two String objects.
6. Define a distance class that has two data fields feet and inches.
 - a. Define a member function to convert distance into meters.
 - b. Two member functions `add_distance(...)` and `sub_distance(...)` to add and subtract distances.
 - c. Another member function to display the distance in proper format.

7. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected. Model this tollbooth with a class called tollBooth.
 - a. The two data items are a type unsigned int to hold the total number of cars, and a type double to hold the total amount of money collected. A constructor initializes both of these to 0.
 - b. A member function called payingCar() increments the car total and adds 0.50 to the cash total.
 - c. Another function, called nopayCar(), increments the car total but adds nothing to the cash total.
 - d. Finally, a member function called display() displays the two totals.
 - e. Make appropriate member functions const.
 - f. Include a program to test this class. This program should allow the user to push one key to count a paying car, and another to count a nonpaying car. Pushing the Esc key should cause the program to print out the total cars and total cash and then exit.
8. Create a class that includes a data member that holds a “serial number” for each object created from the class. That is, the first object created will be numbered 1, the second 2, and so on. To do this, you’ll need another data member that records a count of how many objects have been created so far. (This member should apply to the class as a whole; not to individual objects. What keyword specifies this?) Then, as each object is created, its constructor can examine this count member variable to determine the appropriate serial number for the new object. Add a member function that permits an object to report its own serial number. Then write a main() program that creates three objects and queries each one about its serial number. They should respond I am object number 2, and so on.
9. Implement a stack using classes and objects in C++. The stack class works for integers and implements functions like
 - a. Pop(): returns the top element
 - b. Push(): pushes the element on the top of the stack
 - c. isEmpty(): checks whether the stack is empty or not
 - d. isFull() : checks whether the stack is full or not
 - e. peek(): returns the top of the stack without deleting it
 - f. Inherit this class to create a class called complexStack to store complex numbers as defined in Complex class.

10. Simulate a card game using a class card. The class has two data fields number and suit.
- Write a member function to display the card details. (Remember to display Jack, King, Queen and Ace). Suits will be hearts, diamonds, spades and clubs
 - Randomly assign three cards to the user (use rand() function in cstdlib) and display them as Card 1 is.... Card 2 is ..
 - Swap the cards randomly. 3 moves to swap any two cards. For eg.
 - Move-1: Swap cards 1 and 2
 - Move-2: Swap cards 3 and 1
 - Move-3: Swap cards 3 and 2
 - Ask the user to identify any one card location. Refer figure below for sample outputs.

```
Card 1 is the 7 of clubs
Card 2 is the jack of hearts
Card 3 is the ace of spades
I'm swapping card 1 and card 3
I'm swapping card 2 and card 3
I'm swapping card 1 and card 2
Now, where (1, 2, or 3) is the ace of spades? 1
Sorry, you lose. You chose the 7 of clubs
```

```
Card 1 is the 7 of clubs
Card 2 is the jack of hearts
Card 3 is the ace of spades
I'm swapping card 1 and card 3
I'm swapping card 2 and card 3
I'm swapping card 1 and card 2
Now, where (1, 2, or 3) is the ace of spades? 2
You win!!!!
```